

# System Design Cheat Sheet

The Essential Quick-Reference Guide for Software Engineering Interviews

By Shekhar Patil · Software Engineer · © 2026

FREE EDITION

6 PAGES

50+ CONCEPTS

INTERVIEW-READY

## RESHADED Framework — Ace Any System Design Interview

MUST KNOW

An 8-step structure to tackle any design question confidently. Walk through each letter aloud — interviewers score process as heavily as correctness.

| # | Step         | Time   | What To Say / Do                                                                                                        |
|---|--------------|--------|-------------------------------------------------------------------------------------------------------------------------|
| R | Requirements | 5 min  | Functional & non-functional. "Read or write heavy? Core features? Who are the users?"                                   |
| E | Estimation   | 5 min  | $150\text{M DAU} \times 100 \text{ reads} \div 86,400 = \sim 175\text{K reads/sec}$ . State all assumptions explicitly. |
| S | Storage      | 5 min  | User data → PostgreSQL. Activity logs → Cassandra. Media + blobs → AWS S3.                                              |
| H | High-Level   | 10 min | Draw: Client → CDN → Load Balancer → API Layer → Cache → Primary DB + Read Replicas.                                    |
| A | APIs         | 5 min  | POST /tweets · GET /timeline/{id}?limit=20&cursor;=... State method + path + params.                                    |
| D | Deep Dive    | 10 min | "Let me deep-dive on feed generation — it is the most complex component here."                                          |
| E | Edge Cases   | 3 min  | "Celebrity with 50M followers? Hybrid push-pull instead of pure fan-out on write."                                      |
| D | Decisions    | 2 min  | "I chose Cassandra because writes dominate here — the explicit trade-off is ..."                                        |

## Numbers Every Engineer Must Know Cold

MEMORISE

| Operation               | Latency   | Mental Model                                 |
|-------------------------|-----------|----------------------------------------------|
| L1 cache read           | 0.5 ns    | Baseline — essentially instant               |
| Main memory (RAM)       | 100 ns    | 200× slower than L1 cache                    |
| SSD random read         | 100 µs    | 1,000× slower than RAM                       |
| HDD seek + read         | 10 ms     | 10,000× slower than SSD — avoid on hot paths |
| Intra data-centre RTT   | 0.5 ms    | Same building; fast but not free             |
| Cross-continent RTT     | 150 ms    | Speed-of-light ceiling — serve via CDN       |
| 1M req/day → QPS        | ~12 req/s | Divide daily count by 86,400 seconds         |
| 100M req/day → QPS      | ~1,160/s  | Typical large consumer service               |
| 99.9% uptime (3 nines)  | 8.7 h/yr  | Single redundant server often sufficient     |
| 99.99% uptime (4 nines) | 52 min/yr | Requires active-active multi-zone setup      |

## Technology Selection Guide

RIGHT TOOL FOR THE JOB

Choose datastores based on access pattern, not familiarity. Always justify your decision with an explicit trade-off in the interview.

| Use Case                                    | Technology         | Key Reason to Choose It                            |
|---------------------------------------------|--------------------|----------------------------------------------------|
| Relational data, ACID, complex queries      | PostgreSQL / MySQL | Rich queries, joins, proven ACID compliance        |
| Sub-ms cache, leaderboards, pub/sub         | Redis              | In-memory, ~1 ms, sorted sets, streams             |
| Write-heavy, time-series, IoT at scale      | Apache Cassandra   | LSM-tree writes, linear scale, tunable consistency |
| Full-text and relevance search              | Elasticsearch      | Inverted index, near real-time, faceted search     |
| Event streaming, audit log, async pipelines | Apache Kafka       | Durable, replayable log, millions of events/sec    |
| Object storage — media, backups, ML models  | AWS S3 / GCS       | 11-nines durability, infinite scale, low cost      |
| Global key-value, serverless, auto-scale    | DynamoDB           | Single-digit ms, on-demand capacity, serverless    |
| Distributed locks and leader election       | etcd / ZooKeeper   | Raft consensus, strong CP, coordination primitive  |
| Graph data — social graphs, recommendations | Neo4j / Neptune    | Native graph traversal, Cypher, relationship-first |

## Cache Strategies

READ vs WRITE

| Strategy      | How It Works                                      | Best For                           | Key Risk                                        |
|---------------|---------------------------------------------------|------------------------------------|-------------------------------------------------|
| Cache-Aside   | App checks cache; miss → read DB → populate cache | General purpose — most common      | Stale if DB updated directly; cold-start misses |
| Write-Through | Write to cache + DB synchronously on every write  | Read-heavy; freshness critical     | Slower writes; caches rarely-read data          |
| Write-Behind  | Write cache only; async flush to DB on interval   | Ultra-low write latency (counters) | Data loss if cache crashes before flush         |
| Read-Through  | Cache fetches DB on miss — transparent to app     | Clean app-layer abstraction        | Cache must know DB schema; harder to test       |

## CAP Theorem Decision Tree

PARTITION TOLERANCE IS ALWAYS GIVEN

| When This Is True...                                    | Choose                                         | Real-World Examples                               |
|---------------------------------------------------------|------------------------------------------------|---------------------------------------------------|
| Correctness critical — banking, inventory, reservations | CP (Consistency + Partition Tolerance)         | HBase, ZooKeeper, etcd, MongoDB (majority writes) |
| Availability critical — social feed, shopping cart, DNS | AP (Availability + Partition Tolerance)        | Cassandra, DynamoDB, CouchDB, Riak                |
| Single node — legacy, embedded, prototype               | CA (Consistency + Availability)                | PostgreSQL / MySQL single node (no replication)   |
| Default for most interview designs at 10M+ users        | AP + eventual consistency + compensating logic | Cassandra + read repair + vector clocks + LWW     |

## Rate Limiting Algorithms

| Algorithm              | Mechanism                                        | Pros                                    | Cons                                          |
|------------------------|--------------------------------------------------|-----------------------------------------|-----------------------------------------------|
| Token Bucket           | Tokens refill at rate R; each request consumes 1 | Allows short bursts; memory-efficient   | Burst spike if bucket has accumulated tokens  |
| Leaky Bucket           | Requests drain from FIFO queue at constant rate  | Smooth, predictable output rate         | Bursts queued or silently dropped             |
| Fixed Window Counter   | Count per time window; reset at boundary         | Simplest; very low memory               | "Boundary attack" can double effective limit  |
| Sliding Window Log     | Store exact timestamps; count last N seconds     | Perfectly accurate; no boundary exploit | Memory-intensive at high volume per user      |
| Sliding Window Counter | Weighted blend of current + previous window      | Accurate + memory-efficient             | Slightly approximate — acceptable in practice |

## Essential Design Patterns

MENTION IN DEEP DIVE

| Pattern         | Problem Solved                                         | How It Works                                                                           |
|-----------------|--------------------------------------------------------|----------------------------------------------------------------------------------------|
| Circuit Breaker | Cascading failures when downstream service is down     | Open after N failures; fail-fast fallback; probe with test requests to detect recovery |
| Saga Pattern    | Distributed transactions across multiple microservices | Chain of local DB transactions + events; compensating rollbacks on any failure         |
| CQRS            | Read + write models need different scaling strategies  | Separate read DB (denormalised projections) from write DB; sync via domain events      |
| Outbox Pattern  | Reliably publish events after a DB write without 2PC   | Write event to outbox table atomically; relay polls and publishes asynchronously       |
| Strangler Fig   | Migrate monolith to microservices safely               | Route new traffic to microservice; retire old code path incrementally                  |
| Sidecar         | Cross-cutting concerns duplicated across every service | Deploy sidecar container per service for auth, TLS, tracing (Envoy, Dapr, Linkerd)     |

## Availability Patterns

REDUNDANCY

| Pattern        | When to Use                                            | Concrete Example                                                   |
|----------------|--------------------------------------------------------|--------------------------------------------------------------------|
| Active-Active  | Both nodes serve live traffic; maximise throughput     | Two DB primaries accepting writes with LWW conflict resolution     |
| Active-Passive | Standby promoted on failure; simpler consistency model | MySQL primary + warm replica; 30–60 s automated failover           |
| N+1 Redundancy | N servers needed + 1 spare for instant failover        | 5 app servers + 1 hot standby; replaced on health-check failure    |
| Geo-Redundancy | Survive full region / DC outage; serve from nearest    | Multi-region active-active: us-east-1 + eu-west-1 + ap-southeast-1 |

## Case Study Quick Reference

6 SYSTEMS

These six systems appear in the vast majority of interviews. Know scale numbers, the dominant challenge, and the key technology decision cold.

| System           | Scale                       | Core Challenge                                              | Key Technology Decision                                                    |
|------------------|-----------------------------|-------------------------------------------------------------|----------------------------------------------------------------------------|
| URL Shortener    | 1K writes/s 100K reads/s    | Unique short-code generation; sub-10 ms redirect            | Base62 on auto-increment ID; Redis redirect cache; MySQL mapping           |
| Twitter / X Feed | 3,500 tweets/s 175K reads/s | Fan-out to millions of followers — write time vs. read time | Hybrid push+pull: Redis timelines for regular; pull for celebrity accounts |
| WhatsApp         | 1.2M msgs/s 2B users        | Real-time delivery + reliable offline message buffering     | WebSocket per connection; Erlang BEAM; Kafka; end-to-end encryption        |
| Netflix          | 1B hrs/day 15% global BW    | Adaptive bitrate streaming to 190 countries at low latency  | Open Connect CDN (ISP-embedded); ABR (HLS/DASH); S3 video chunks           |
| Uber             | 19M trips/day <2 s matching | Real-time geospatial driver-rider matching at city scale    | Redis GEO + Geohash; ETA-based ranking; Kafka trip state machine           |
| Google Drive     | 1B users Petabytes          | Multi-device sync + deduplication + conflict resolution     | Chunked upload (4 MB); SHA-256 dedup; CRDTs for collaborative editing      |

## The 10 Golden Rules of System Design

ALWAYS TRUE

|   |                                                                                                  |    |                                                                                               |
|---|--------------------------------------------------------------------------------------------------|----|-----------------------------------------------------------------------------------------------|
| 1 | <b>Clarify before drawing.</b> Missing one constraint invalidates the whole design.              | 2  | <b>Estimate first, design second.</b> Numbers reveal whether your choice actually scales.     |
| 3 | <b>Start simple, earn complexity.</b> Propose the naive solution before adding Kafka.            | 4  | <b>Stateless services scale.</b> Push all mutable state to purpose-built data stores.         |
| 5 | <b>Cache the hot 20%.</b> Not everything belongs in Redis — be selective.                        | 6  | <b>Async non-critical paths.</b> Message queues absorb traffic spikes gracefully.             |
| 7 | <b>Design for failure.</b> Retries, timeouts, circuit breakers, fallbacks.                       | 8  | <b>State trade-offs explicitly.</b> "I chose X over Y because Z" impresses every interviewer. |
| 9 | <b>Horizontal &gt; vertical scale.</b> Add commodity servers; you cannot infinitely upgrade one. | 10 | <b>Observability is not optional.</b> No dashboards + no alerts = flying blind in production. |

### Database Sharding Strategies

NEW

| Strategy           | How It Works                                        | Good For                                       | Key Pitfall                                            |
|--------------------|-----------------------------------------------------|------------------------------------------------|--------------------------------------------------------|
| Range-Based        | Shard by value range — A–M → shard 1, N–Z → shard 2 | Range queries; ordered time-series scans       | Hot spots if key distribution is skewed                |
| Hash-Based         | shard_id = hash(key) % N — even distribution        | Uniform writes; user IDs; session tokens       | No range queries; full reshard on resize               |
| Consistent Hashing | Keys + nodes on virtual ring; clockwise routing     | Caches, CDNs; minimal movement on scale-out    | Virtual nodes needed for even distribution             |
| Directory-Based    | Lookup table maps each key to its shard             | Flexible; routing logic can change dynamically | Lookup table = SPOF + read bottleneck                  |
| Geo-Sharding       | Route users by geography to nearest shard           | GDPR compliance; latency; data sovereignty     | Cross-region queries expensive; travel breaks locality |

### Indexing Strategies

| Index Type       | How It Works                                       | Use When                                       | Trade-Off                                    |
|------------------|----------------------------------------------------|------------------------------------------------|----------------------------------------------|
| B-Tree (default) | Balanced tree; O(log n); supports range scans      | Most queries; primary + foreign keys; ORDER BY | Write overhead on every insert/update        |
| Hash Index       | O(1) exact-match via hash map                      | Equality comparisons only — sessions, caches   | No range scans; low-cardinality cols         |
| Composite Index  | Index on (A,B,C); left-prefix rule applies         | Multi-column WHERE in known column order       | Column order critical; leftmost cols only    |
| Partial Index    | Index only rows matching a WHERE condition         | High-cardinality subsets: e.g., active orders  | Query WHERE must match index filter exactly  |
| Covering Index   | Includes all cols needed — enables index-only scan | Read-heavy hot tables; avoid heap fetches      | Larger index; extra cols increase write cost |
| Full-Text Index  | Inverted token index; keyword search + ranking     | Search boxes, autocomplete, document retrieval | Not for exact-match; refresh on updates      |

### ACID vs BASE — The Fundamental Trade-off

#### ACID — PostgreSQL, MySQL

| Property    | ACID Guarantee                                   |
|-------------|--------------------------------------------------|
| Atomicity   | All-or-nothing; no partial writes ever committed |
| Consistency | DB always transitions between valid states       |
| Isolation   | Concurrent txns cannot see each other's state    |
| Durability  | Committed data survives crashes and power loss   |

Use when: payments, bookings, inventory — wrong data = real harm.

#### BASE — Cassandra, DynamoDB

| Property              | BASE Meaning                                      |
|-----------------------|---------------------------------------------------|
| Basically Available   | System stays up even during partial failures      |
| Soft State            | State may change without input (replication lag)  |
| Eventually Consistent | All replicas converge to the same value over time |

Use when: social feeds, analytics — stale reads acceptable.

## Microservices vs Monolith

| Dimension     | Monolith                                | Microservices                                     |
|---------------|-----------------------------------------|---------------------------------------------------|
| Team size     | Small (1–10 devs); shared codebase      | Multiple teams; each owns + deploys one service   |
| Deployment    | Single artefact; simple CI/CD           | Independent deploys per service; needs Kubernetes |
| Data layer    | Shared DB; easy joins across entities   | Each service owns its DB; no cross-service joins  |
| Failure scope | One bad deploy can take down everything | Failures contained; other services keep running   |
| Network cost  | In-process calls; microsecond overhead  | 0.5–5 ms per hop; multiplies across call chains   |
| When to use   | MVP, startup, <100K users, one team     | Proven product, 10M+ users, independent scaling   |

## API Design Best Practices

REST &amp; gRPC

| Principle        | Do This                                                 | Avoid This                                      |
|------------------|---------------------------------------------------------|-------------------------------------------------|
| Versioning       | PUT /v1/users/{id} — version in URL or Accept header    | Breaking changes to existing endpoint URL       |
| Idempotency      | PUT+DELETE idempotent; POST uses Idempotency-Key header | Retry POST payment without key — duplicates     |
| Pagination       | Cursor-based: ?limit=20&cursor;=                        | OFFSET — breaks on large datasets; O(n) scans   |
| Status codes     | 200 OK · 201 Created · 400 Bad Request · 429 Too Many   | Always 200 with error buried in JSON body       |
| Rate-limit hints | X-RateLimit-Limit · Remaining · Retry-After             | Silent 429 — no Retry-After, clients spam retry |

## Fatal Interview Mistakes — And How to Avoid Them

AVOID

|                                           |                                                                                                     |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------|
| ✗ Jump to solution without clarifying     | Always ask requirements first. Ambiguity is deliberate — interviewers reward good questions.        |
| ✗ Design for infinite scale from minute 1 | Start simple. Add complexity only when estimates justify it — not as a reflex.                      |
| ✗ Stay silent while designing             | Think out loud. Interviewers cannot evaluate reasoning they cannot hear — silence kills your score. |
| ✗ Ignore failure modes entirely           | Every component fails. Address proactively: "If the cache goes down, here is my fallback."          |
| ✗ Pick tech before understanding the data | Understand access patterns first. Technology choice must flow from requirements, not habit.         |
| ✗ Skip non-functional requirements        | Ask for latency SLA, availability target (99.9% vs 99.99%), and consistency model upfront.          |

Want the complete, in-depth version?

© 2026 Shekhar Patil. Free for personal use. Redistribution without written permission is prohibited.